Early-Stage Detection of Encrypted Malware Traffic via Multi-Flow Temporal Graph Learning

Jizhe Jia, Yi Zhao, *Member, IEEE*, Meng Shen, *Member, IEEE*, Susu Cui, Jing Wang, Bufan Zhao, Wei Wang, *Member, IEEE*, and Liehuang Zhu, *Senior Member, IEEE*

***Abstract*—Malware widely adopts network traffic encryption techniques to conceal malicious activities. Recent research has demonstrated the effectiveness of machine learning (ML)-, deep learning (DL)-, and pre-training-based malware traffic detection methods. However, a vast majority of these methods rely on the collected complete traffic during the malware attack. While certain methods can operate on partial traffic, their detection accuracy often significantly decreases when the available data is restricted to the extreme early stage, where information is most sparse. In this paper, we propose *DawnGuard*, an effective early-stage encrypted malware traffic detection framework through multi-flow temporal graph learning. Specifically, based on the temporal packet density distribution analysis, *DawnGuard* innovatively proposes a self-adjusting data augmentation strategy for early-stage malware traffic, which can force the model to focus on the early-stage interaction phase with more distinguishable properties. Meanwhile, considering that temporal-topological correlations among multiple flows can provide more distinguishable properties in a malware attack, we further develop a temporal graph learning framework to extract features, which can form *Multi-Flow Graph Features (MGF)*. By utilizing *MGF*, *DawnGuard* implements a Vision Transformer-based detection mechanism, enabling accurate and precise encrypted malware traffic detection with early-stage traffic by capturing both local and global contextual relationships. Extensive experiments with two real-world datasets demonstrate that *DawnGuard* outperforms the state-of-the-art (SOTA) methods in three typical scenarios: varying early-stage time windows, imbalanced data, and unseen malware detection. Particularly, *DawnGuard* achieves an average F1 of 95.11%, 8.7% higher than the SOTA method, by only utilizing the first 20% loading ratio of complete traffic.**

***Index Terms*—Malware traffic detection, encrypted traffic analysis, graph learning.**

Received 19 May 2025; revised 12 January 2026 and 24 March 2026; accepted 12 April 2026. Date of publication 17 April 2026; date of current version 4 May 2026. This work was supported in part by NSFC under Project U25A20428, Project U23A20304, and Project 62472036; in part by Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing; and in part by Beijing Nova Program. The associate editor coordinating the review of this article and approving it for publication was Dr. Daisuke Mashima. *(Corresponding author: Meng Shen.)*

Jizhe Jia and Meng Shen are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China, and also with the State Key Laboratory of Cryptology, Beijing 100878, China (e-mail: jiajizhe@bit.edu.cn; shenmeng@bit.edu.cn).

Yi Zhao, Bufan Zhao, and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: zhaoyi@bit.edu.cn; bitzbf@163.com; liehuangz@bit.edu.cn).

Susu Cui is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100089, China (e-mail: cuisusu@iie.ac.cn).

Jing Wang is with the National Computer Network Emergency Response Technical Team/Coordination Center of China, Beijing 100094, China (e-mail: wangjing_str@163.com).

Wei Wang is with the Key Laboratory for Intelligent Networks and Network Security, Ministry of Education, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: wangwei1@bjtu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3685079

I. Introduction

THE proliferation of network encryption protocols (SSL/TLS [1]), while essential for privacy protection, has inadvertently empowered cyber attackers to weaponize cryptographic channels. Modern malware, particularly remote access trojans (RATs) that grant attackers administrative control over compromised systems, increasingly leverages encrypted traffic to evade traditional signature-based network intrusion detection systems (NIDS) [2], [3], [4], [5]. This encryption paradigm fundamentally reduces observable payload information at the transport layer [6], [7], [8], [9], resulting in sparser features available for detection. Alarmingly, 60% of malware spread over encrypted connections during *Q*4 of 2024, which is an 8% increase from last quarter and a continued increase for the year [10].

Researchers have proposed various detection methods that can be categorized into four types based on their technical approaches: rule-based methods [11], machine learning (ML)-based methods [2], [4], [12], deep learning (DL)-based methods [3], [13], [14], and pre-training-based methods [15], [16], [17]. Most of these methods mainly focus on specific field features, statistical features, and packet sequence features of full flows to build detection models. However, malware attacks typically unfold through a phased and sequential process, and waiting for a complete flow collection will result in delayed detection, which introduces a prolonged period before taking countermeasures, resulting in non-negligible security risks (e.g., financial losses and confidential data thefts). For instance, in 2022, Costa Rica declared a nationwide emergency following a major ransomware attack by the Conti Group, impacting its Department of Finance and business activities. The ransom demanded up to 20 million [18]. Early-stage malware traffic detection refers to a continuous monitoring and detection mechanism that can detect malware traffic as early as possible (i.e., 20%, 30%) before the complete flow transmission, which mitigates the risks to the victim, including system file encryption, credential theft, and data exfiltration.

Despite some progress on detection accuracy and efficiency in encrypted malware traffic detection [2], [3], [4], critical challenges remain, which can be summarized as follows: **1) Information sparsity in early-stage traffic.** While certain methods [14], [16], [20] have explored detection using partial traffic features, they heavily rely on the temporal accumulation of intra-flow features. For instance, sequence-based models like RNNs with attention [14] often require a substantial observation budget (e.g., a large number of initial packets or higher loading ratios) to achieve stable performance. When the available traffic is extremely limited (e.g., the first 20%), the information within a single isolated flow is often too sparse

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FLOW DURATION STATISTICS (SECONDS) OF THE SIX TYPICAL MALWARE IN THE CTU-MALWARE DATASET [19]

Malware	Mean	Median	Max
Dridex	28.17	1.21	3,724.45
Emotet	120.91	4.71	903.76
Geodo	180.51	9.00	1,065.92
Miuref	345.79	9.00	422,062.67
Zeus	16.20	0.74	8,632.08
TrickBot	62.74	65.42	1,394.44

to provide a definitive signature, leading to a significant drop in accuracy for these methods. **2) Heterogeneity of attack patterns.** The heterogeneity of malware attack patterns is a natural concealment method relative to detection methods, thus posing a critical challenge to achieving high precision in early-stage traffic detection. For example, malicious activities (e.g., victim scanning, spam propagation, or click fraud) exhibit highly divergent flow duration characteristics [21]. Botnets like Zeus predominantly generate short-lived flows (median duration < 1 second), whereas advanced threats like TrickBot sustain prolonged connections (median duration > 1 minute), as quantified in TABLE I.

In this paper, we propose *DawnGuard*, an effective early-stage detection method that can accurately distinguish encrypted malware traffic based on early-stage traffic. Specifically, due to information sparsity, existing single-flow detection methods fail to identify encrypted malware traffic in the early stage [20], [22]. To maximize the utilization of available information in the early stage, we can analyze multi-flow temporal interaction patterns that reveal distinct attack behaviors (e.g., abnormal spam bot-SMTP communications [4]) that deviate from benign traffic.

DawnGuard is composed of three key modules. *First*, the early-stage interaction phase (e.g., protocol handshake and configuration download) contains more distinctive properties than the subsequent phase [2], [23]. To force the model to focus on the early-stage interaction phase, we design a self-adjusting data augmentation strategy. Crucially, this strategy is self-adjusting because it dynamically determines the optimal cut-off point for tail-masking for each flow by analyzing its unique temporal packet density distribution. This ensures that the augmented data consistently retains the most discriminative, the early-stage interaction phase of the traffic, which systematically addresses the sparsity of early-stage traffic information. *Second*, to address attack pattern heterogeneity, we design a multi-flow temporal graph learning algorithm, which can explore temporal and topological correlations between multiple flows to enrich discriminative features in a malware attack, named *Multi-Flow Graph Features (MGF)* from multiple flows. Notably, compared with single flow-based detection methods, a more global view based on multiple flows within a malware attack reflects more flow inter-relationships. *Finally*, we construct a Vision Transformer (ViT)-based detector, which can capture both local and global contextual relationships for accurate and precise encrypted malware traffic detection with early-stage traffic. For real-world deployments, *DawnGuard* employs a persistent monitoring mechanism that iteratively collects packets and performs progressive traffic analysis across subsequent monitoring cycles.

The main contributions of this paper are as follows:

- We propose *DawnGuard*, an effective early-stage malware traffic detection framework based on multi-flow temporal graph learning. By focusing on information sparsity and attack pattern heterogeneity, it achieves high-precision early-stage detection by emphasizing high-density information, expanding multi-flow distinguishable properties, as well as integrating local and global contextual relationships.
- To solve information sparsity and boost training data diversity, we propose a self-adjusting data augmentation strategy, which combines temporal packet density analysis with a tail-masking method, thereby forcing the model to focus on the early-stage interaction phase with more distinctive characteristics.
- To adapt to the heterogeneity of attack patterns, we develop the temporal graph learning method to extract temporal-topological correlations among multiple flows in a malware attack (i.e., *MGF*), which can provide more distinguishable properties for early-stage detection.
- We evaluate *DawnGuard* in two real-world datasets in three typical scenarios, i.e., varying early-stage time windows, imbalanced data, and unseen malware detection. The results demonstrate that *DawnGuard* significantly outperforms existing methods in all scenarios. Particularly, *DawnGuard* achieves an average F1 of 95.11%, 8.7% higher than the SOTA method, with the first 20% loading ratio of complete traffic. We release the dataset and source code of *DawnGuard*.[1]

The rest of the paper is organized as follows. We first introduce the background and the threat model in Section II. We present the high-level design of *DawnGuard* in Section III and the design details in Section IV. Next, we conduct the theoretical analysis in Section V and comprehensive experiments to evaluate the detection performance of *DawnGuard* in Section VI. Finally, we review related works in Section VII and conclude this paper in Section VIII.

II. PRELIMINARIES

In this section, we introduce the threat model and the definition of early-stage detection.

A. Threat Model

From the attacker's perspective, the attack surface comprises the network boundary where compromised internal devices interact with external malicious infrastructures, such as C&C servers and malware delivery sites [2]. We assume that the attacker can leverage encryption protocols (e.g., SSL/TLS) to obscure payloads, rendering traditional Deep Packet Inspection (DPI) ineffective. In addition, they can coordinate multiple network flows to execute sophisticated, multi-stage objectives. Note that, these attacks are typically constructed with numerous addresses controlled by attackers [4].

From the defender's perspective, *DawnGuard* operates at the network level, collecting traffic via port mirroring at edge routers without deploying agents on end-user devices

[1] https://github.com/jiajizhe1997/DawnGuard

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

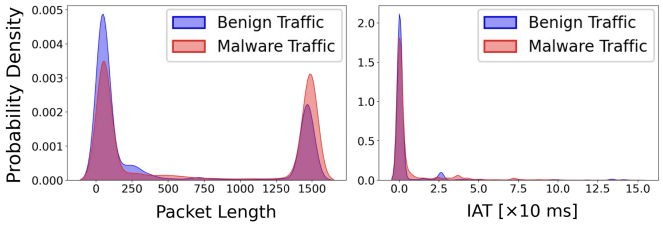


Fig. 1. The KDE of malware and benign traffic across packet length and IAT in CTU-Malware dataset [19].

[3]. Our detection model operates under the assumptions that the encryption algorithms can not be decrypted. Furthermore, our detection scope is focused on detecting active network attacks, excluding passive eavesdropping or side-channel analysis which do not generate network traffic.

B. Early-Stage Detection

A core challenge in this threat landscape is achieving effective early-stage detection, the ability to identify and prevent damage before an attack sequence fully unfolds. Ideally, a detection model should react within the initial phase of a connection (e.g., the first 20-40% of the complete traffic).[2] This not only enables timely mitigation but also ensures the model acquires sufficient discriminative data before a potentially short-lived malicious flow terminates [24].

A natural question that arises is the selection between relative and absolute early-stage time windows. We analyze the flow durations of six representative malware families (i.e., Dridex, Emotet, Geodo, Miuref, Zeus, and TrickBot) from the malware capture project [19]. As shown in TABLE I, the flow durations exhibit significant statistical variations across different malware. An absolute window that is effective for a short-lived attack might be insufficient for a long-running one. Therefore, to ensure that *DawnGuard* remains effective across diverse malware, we adopt a relative early-stage time window, analyzing packets across different loading ratios of a flow rather than relying on a fixed absolute time window.

III. Design of DawnGuard

In this section, we present the motivation for our design and provide an overview of *DawnGuard*.

A. Motivation

Malware traffic detection faces a dual challenge. As shown in Fig. 1, fundamental single-flow features (e.g., packet length, IAT) exhibit significant overlap between benign and malware traffic in CTU-Malware [19]. This ambiguity limits single flow-based methods [15], [25], [26], which lack the context to detect multi-flow patterns like C&C communication.

To incorporate such context, graph-learning techniques [2], [4], [27] have been adopted but suffer from two shortcomings. *First*, global modeling paradigms indiscriminately include all concurrent flows, introducing noise from irrelevant activities. *Second*, static graph modeling ignores temporal interdependencies, causing flows with different chronological orders to yield isomorphic structures, thus eroding the detection of time-sensitive patterns.

[2]For instance, *DawnGuard* detects TrickBot traffic, which has an average duration exceeding 60 seconds, using only the first ~12 seconds (20%) of the flow, as shown in TABLE II.

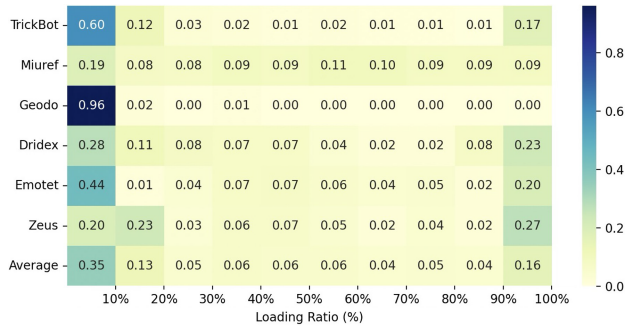


Fig. 2. Packet load ratio analysis over time for the flows of six typical malware in the CTU-Malware dataset [19].

Afterward, we explore the possibility of achieving early-stage malware traffic detection without waiting for the whole flow termination. While recent methods [14] have investigated early detection by focusing on initial packet segments within isolated flows, they remain constrained by single-flow information scarcity. Our analysis of such methods reveals that they often require a relatively high observation budget (e.g., a substantial number of initial packets or higher loading ratios) to extract stable signatures. When the available traffic is extremely limited (e.g., the first 20%), the internal information of a single flow is often too sparse to provide a definitive malicious signature, leading to degraded reliability. However, temporal packet density analysis (Fig. 2) reveals that even within these sparse segments, early interactions (e.g., handshakes, C&C establishment) exhibit higher packet concentrations and more distinctive properties than subsequent phases. Thus, the key challenge is to extract these discriminative signals from minimal and sparse early-stage data.

Driven by these observations, the design of *DawnGuard* is guided by two core intuitions to overcome the limitations of current detection approaches. *First*, to resolve single-flow ambiguity and filter global noise, we propose a multi-flow temporal graph learning framework. Unlike single-flow sequence models that hit an information bottleneck in the earliest phases, this framework leverages the topological structure of concurrent flows to reconstruct the global attack context, thereby isolating attack progression from background noise even with minimal data. *Second*, to specifically address information sparsity at extremely low loading ratios, we design a self-adjusting data augmentation mechanism. By dynamically locating high-density interaction phases and generating diverse, realistic training samples, this mechanism forces the model to learn intrinsic malicious patterns that remain invisible to methods relying on isolated single flow or static observation windows.

B. Overview of DawnGuard

As shown in Fig. 3, itDawnGuard consists of three modules: Self-Adjusting Data Augmentation, Multi-Flow Temporal Graph Learning, and Early-Stage Malware Detection.

1) Self-Adjusting Data Augmentation: To address information sparsity, we design a self-adjusting mechanism that dynamically determines the optimal tail-masking point for each flow. This approach generates diverse training samples

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

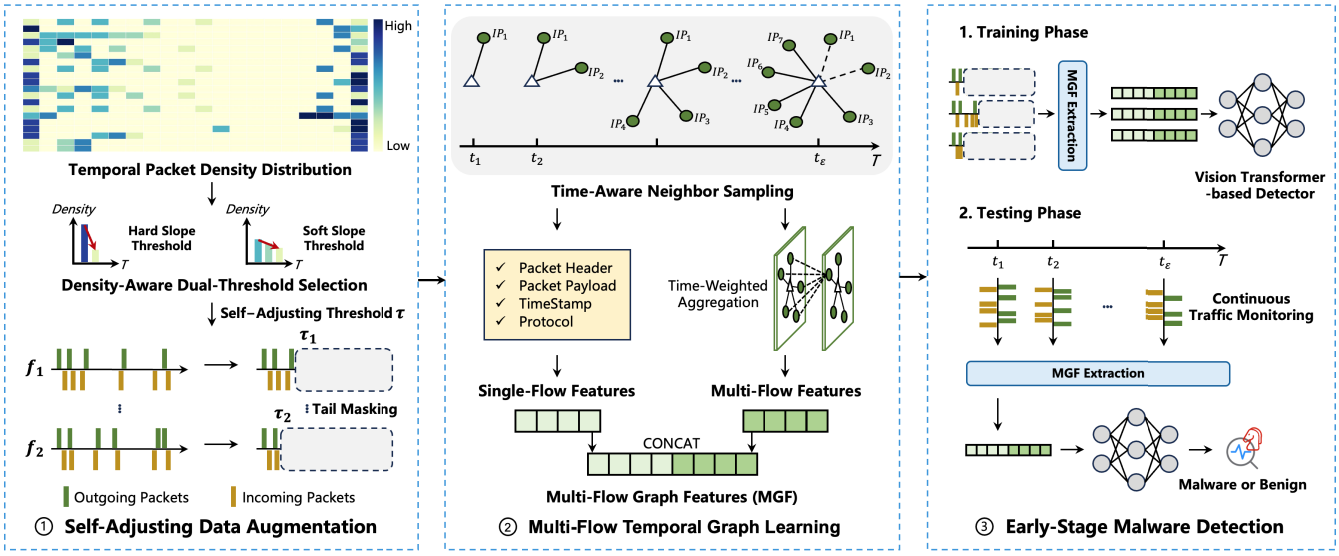


Fig. 3. The system overview of *DawnGuard*.

and directs the model's attention to the critical early interaction phase with distinctive characteristics. We detail this module in Section IV-A.

2) Multi-Flow Temporal Graph Learning: To address attack pattern heterogeneity, we design an algorithm to synergistically explore temporal-topological correlations among multiple flows (i.e., *MGF*). This approach integrates distinguishable properties reflected by flow inter-relationships to enhance early-stage malware detection. We elaborate on this module in Section IV-B.

3) Early-Stage Malware Detection: Given the multi-flow-based traffic representation, i.e., MGF of each flow, we develop a ViT-based detector to capture both local and global contextual relationships from spatial-temporal features and detect early-stage encrypted malware traffic accurately and precisely. We will describe the details of our detector in Section IV-C.

IV. Design Details

In this section, we present the design details of *DawnGuard*, including self-adjusting data augmentation, multi-flow temporal graph learning, and early-stage malware detection. The main notations are illustrated in Table II.

A. Self-Adjusting Data Augmentation

To address the challenge of information sparsity in early-stage traffic, we propose a self-adjusting data augmentation module. Self-adjusting refers to the mechanism that dynamically determines the optimal tail-masking point for each traffic flow, rather than applying a fixed or random masking ratio. This process aims to precisely preserve the critical, high-density early-stage interaction phase (e.g., protocol handshakes) within the augmented traffic. Consequently, this strategy forces the model to focus on the more distinguishable property part in the early stages, thereby boosting training data diversity and enhancing detection performance.

1) Temporal Packet Density Distribution Analysis: To ensure the augmented traffic contains sufficient information for detection, we analyze packet distributions by dividing flows into loading ratio slots. We define the packet density $d_i \in D = \{d_1, d_2, \ldots, d_n\}$ as the ratio of packets in the i-th slot to the total packet count. For generality, we analyze six typical malware from the CTU-Malware dataset [19].

As illustrated in Fig. 2, packet density is significantly higher in the early-stage interaction phase than in subsequent

TABLE II
List of Notations

Notation	Meaning
Self-Adjusting Data Augmentation	
$D, \tilde{D}[k]$	Original and smoothed packet density set of the traffic
F	A set of flows
d_i	The packet density of the i-th time slot
τ_h, τ_s	Hard and soft thresholds for the termination of the early-stage interaction phase
θ	Sensitivity hyperparameter controlling the soft threshold
γ	Peak coefficient for hard threshold calculation
β	Search range for hard threshold calculation
α	Minimum retention ratio for tail masking
Multi-Flow Temporal Graph Learning	
$\mathcal{G}$	Bipartite temporal graph $\mathcal{G} = (\mathcal{H}, \mathcal{S}, \mathcal{E}, \mathcal{F}, \mathcal{T})$
$\mathcal{H}, \mathcal{S}$	A vertex set of hosts, external servers
$\mathcal{E}$, $\mathcal{F}$	Edge and feature set of flows between hosts and servers
M	Number of packets for *PBM*
X, Y	Number of header bytes, and payload bytes for *PBM*
C_e	Collection of edges sharing the same host with edge e
f_e	Single-flow feature vector (*PBM*) of edge e
τ^k	Time window threshold for neighbor filtering at layer k
n	Maximum number of sampled neighbors per edge
λ	Time decay factor for temporal attention weighting
$\mathbf{h}_e^{(k)}$	Latent representation of edge e at the k-th layer
$\mathbf{W}_{agg}^{(k)}$	Learnable aggregation weight matrices at layer k
$\mathbf{W}_{self}^{(k)}$	Learnable self-loop weight matrices at layer k
z_e	Final multi-flow feature embedding
Early-Stage Malware Detection	
$\mathcal{M}$	Reshaped 2D input matrix for the ViT-based detector
Q, L	The length of *PBM* and z_e
h	Number of attention heads in Multi-Head Self-Attention
$W^{\{Q,K,V,O\}}$	Learnable projection matrices for query, key, value, and output in MHSA
$W_{\{1,2\}}$	Weight parameters for the Feed-Forward Network
$b_{\{1,2\}}$	Bias parameters for the Feed-Forward Network

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

phases. Specifically, 35% of total packets (and up to 96% for Geodo) are typically transmitted within the initial 10% of flow duration, driven by protocol handshakes and C&C establishment. This pattern concludes with a low-activity phase or session termination. Consequently, we design a mask-based data augmentation strategy that preserves these high-density, discriminative early-stage interactions.

2) Mask-Based Data Augmentation: Data augmentation improves model performance by boosting training diversity, but fixed masking ratios either erase critical semantics or retain redundant noise. To address this, we propose a density-aware, self-adjusting data augmentation strategy. By dynamically generating early-stage samples with adaptive tail-masking, it enables the subsequent graph learning module to capture multi-scale temporal dependencies and resolve information sparsity. As detailed in Algorithm 1, the process involves three phases: adaptive time bin partition, density-aware dual-threshold selection, and adaptive tail masking.

a) Adaptive Time Bin Partition (Algorithm 1, Lines 3-8). To handle flows of varying durations, we partition each flow into K adaptive time bins and calculate the raw packet density array D in each time bin in Eq. (1).

$$\mathcal{B} = \{b_k\}_{k=1}^{K+1} \text{ where } b_k = t_1 + \frac{(k-1)}{K} T_{\text{total}} \tag{1}$$

We then apply Savitzky-Golay smoothing [28] to suppress high-frequency noise while preserving critical phase transition patterns, yielding the smoothed density curve $\tilde{D}$, thereby facilitating the robust location of density plunge points for threshold calculation in Eq. (2).

$$\tilde{D}[k] = \text{SG-Filter}(D, \textit{window}, \text{poly_order}) \tag{2}$$

Here, *window* denotes the number of adjacent data points used for local polynomial fitting, while *poly_order* denotes the polynomial degree to model the underlying trend.

b) Density-Aware Dual-Threshold Selection (Algorithm 1, Lines 10-16). We design a dual-threshold mechanism to locate the optimal plunge point of early-stage interactions. A hard threshold τ_h detects sharp traffic drops (e.g., handshake completion) by finding the first bin where density falls below a fraction γ of the peak density within a search bound β:

$$k_h = \min \left\{ k \in \{1, \ldots, K\} \mid \tilde{D}[k] < \gamma \cdot \max(\tilde{D}[1 : \lceil \beta K \rceil]) \right\} \tag{3}$$

Alternatively, a soft threshold τ_s identifies progressive density decay. We introduce a sensitivity hyperparameter θ (i.e., representing the declination angle) to control this progressive decay detection. The soft threshold is dynamically calculated based on the average density scaled by this slope factor, as defined in Eq. (4).

$$\tau_s = \tan(\theta) \cdot \frac{1}{K} \sum_{k=1}^{K} \tilde{D}[k] \tag{4}$$

Here, $\tilde{D}[k]$ represents the packet density in the k-th bin, and $\tan(\theta)$ adjusts the threshold relative to the overall traffic volume. We then slide a three-slot window to find the first point k_s where the density decrease Δ_s across three consecutive bins consistently exceeds τ_s:

$$k_s = \min \left\{ k \in \{3, \ldots, K\} \mid \bigwedge_{j=0}^{2} (\Delta_s[k-j] > \tau_s) \right\} \tag{5}$$

Algorithm 1 Self-Adjusting Data Augmentation

Input: Flow set $F = \{f_1, f_2, \ldots, f_n\}$; Flow $f = \{p_1, p_2, \ldots, p_N\}$ with timestamps $\{t_i\}_{i=1}^{N}$; Minimum retention ratio α; Minimum packet constraint μ; Hard threshold coefficient γ; Peak density search upper bound β; Soft threshold angle θ

Output: Masked flow set F_{masked}

1 **for** *each flow* $f_i \in F$ **do**
2 // **Phase 1: Adaptive Time Bin Partition**
3 $T_i \leftarrow \text{duration}(f_i)$
4 $K \leftarrow \min(100, \max(20, \lfloor N/5 \rfloor))$
5 $\mathcal{B} \leftarrow \{b_k\}_{k=1}^{K+1}$ where $b_k = t_1 + \frac{(k-1)}{K} T_{\text{total}}$
6 $D[k] \leftarrow \sum_{i=1}^{N} \mathbb{I}(b_k \le t_i < b_{k+1}), \quad \forall k \in \{1, \ldots, K\}$
7 $\tilde{D}[k] \leftarrow \text{SG-Filter}(D, \text{window}, \text{poly_order})$
8 ▷ Eliminate density noise
9 // **Phase 2: Density-Aware Dual-Threshold Selection**
10 $\tau_h \leftarrow \gamma \cdot \max(\tilde{D}[1 : \lceil \beta K \rceil])$ ▷ Compute hard threshold
11 $k_h \leftarrow \min \left\{ k \in \{1, \ldots, K\} \mid \tilde{D}[k] < \tau_h \right\}$
12 $\tau_s \leftarrow \tan(\theta) \cdot \frac{1}{K} \sum_{k=1}^{K} \tilde{D}[k]$ ▷ Compute soft threshold
13 $\Delta_s[k] \leftarrow \tilde{D}[k] - \tilde{D}[k+1]$
14 $k_s \leftarrow \min \left\{ k \in \{3, \ldots, K\} \mid \bigwedge_{j=0}^{2} (\Delta_s[k-j] > \tau_s) \right\}$
15 $k_{\text{thre}} \leftarrow \min(k_h, k_s)$ if $\{k_h, k_s\} \neq \emptyset$
16 $k_{\text{thre}} \leftarrow \lfloor \alpha K \rfloor$ otherwise
17 // **Phase 3: Adaptive Tail Masking**
18 $k_{\text{final}} \leftarrow \max(k_{\text{thre}}, \lceil \alpha K \rceil, \lceil \frac{\mu}{N} K \rceil)$
19 ▷ Select optimal cut-off point
20 $t_{\text{mask}} \leftarrow b_{k_{\text{final}}}$
21 $W_i \leftarrow [t_{\text{mask}}, T_i]$
22 Apply tail masking to f_i over W_i
23 **return** F_{masked}

The final termination point is determined by the minimum of k_h, k_s, and a pre-defined minimum retention bound derived from ratio α (Algorithm 1, lines 15-16).

c) Adaptive Tail Masking (Algorithm 1, Lines 18-22). Based on the identified termination point, we establish a masking window W_i to obscure the non-critical tail traffic of flow f_i, yielding F_{masked} (Algorithm 1, lines 18-22). This strategic truncation forces the subsequent graph learning module to focus on highly distinguishable early-stage interaction representations.

B. Multi-Flow Temporal Graph Learning

Utilizing the early-stage traffic generated by the self-adjusting data augmentation module, the multi-flow temporal graph learning module explores temporal-topological correlations among multiple flows in a malware attack (i.e., *MGF*), which enriches discriminative features for early-stage malware traffic detection. Specifically, we first design a traffic representation method to extract single-flow features (i.e., *Packet Byte Matrix*) efficiently from raw traffic. Afterward, we construct a host-server bipartite graph to capture the interactions between hosts and external servers. Based on the bipartite graph, we design a multi-flow temporal graph learning algorithm to extract temporal-topological correlations between multiple flows in a malware attack. The details are as follows:

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

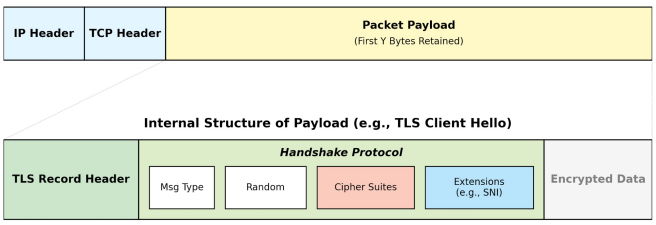


Fig. 4. Illustration of observable unencrypted metadata (e.g., Extensions) within the payload bytes of encrypted traffic.

1) Single-Flow Feature Extraction: To extract multi-dimensional features efficiently, we design the *Packet Byte Matrix (PBM)*. For each flow, we strictly monitor the first M consecutive packets. Each packet is partitioned into a header segment (X bytes) and a payload segment (Y bytes). To prevent the model from overfitting to network-specific artifacts (e.g., MAC/IP addresses and port numbers), we apply standard traffic anonymization strategies [29] to these raw bytes.

We preserve complete header fields and retain the first Y bytes of the packet payload. Crucially, while encryption obscures the semantic content of the payload, the initial bytes in early-stage traffic often carry unencrypted protocol metadata specific to malware families [2], [20], such as TLS handshake extensions and Server Name Indication (SNI), as shown in Fig. 4. By including these payload segments in the *PBM*, we enable the model to capture these fingerprints that remain observable even in encrypted traffic. The header-payload sequences from M packets are reshaped into a 2D matrix $M \in \mathbb{R}^{2\times Q}$. Here, $Q = M \times (X + Y)/2$.

2) Bipartite Graph Construction: We design a bipartite graph, $\mathcal{G} = (\mathcal{H}, \mathcal{S}, \mathcal{E}, \mathcal{F}, \mathcal{T})$ to represent the interaction relationships between hosts and external servers that are linked to each host. We denote the hosts as a vertex set $\mathcal{H}$, where each host is uniquely identified by its IP address. Differently, we denote the external servers as a vertex set $\mathcal{S}$, where each external server is uniquely identified by its IP address and port. Besides, we adopt an edge set $\mathcal{E} = \{e|e = \{h_e, s_e\}, \forall h_e \in \mathcal{H}, \forall s_e \in D\}$ to denote all interaction relationships between hosts and servers. If there is a connection between the host h and an external server s, then an edge $e = (h, s)$ is constructed between them. We denote the features extracted from every single flow e as $\mathcal{F} = \{f_e|\forall e \in \mathcal{E}\}$ through the single-flow feature extraction. We denote the corresponding timestamp of every flow e as $\mathcal{T} = \{t_e|\forall e \in \mathcal{E}\}$. Here, t_e is defined as the timestamp of the first packet in flow e.

3) Graph Representation Learning: To explore temporal-topological correlations among concurrent flows in a malware attack, we design a *Temporal Graph Learning Algorithm* tailored for early-stage detection. Compared to traditional Probabilistic Graphical Models (PGM) [30] that rely on predefined assumptions and struggle with continuous traffic, our approach enables joint end-to-end optimization of intra-flow features and inter-flow topologies, effectively capturing complex non-linear malware attack correlations.

As outlined in Algorithm 2, the learning process integrates sinusoidal temporal encoding, time-aware neighbor sampling, and time-weighted aggregation across K layers to generate the final *Multi-Flow Graph Features (MGF)*.

Algorithm 2 Temporal Graph Learning

Input: Graph $\mathcal{G} = (\mathcal{H}, \mathcal{S}, \mathcal{E}, \mathcal{F}, \mathcal{T})$; $\mathcal{F} = \{f_e \mid \forall e \in \mathcal{E}\}$; $\mathcal{T} = \{t_e \mid \forall e \in \mathcal{E}\}$; Depth K; Edge neighborhood collection $C_e = \{u_{h,s} : (h_u, s_u) \mid u_{h,s} \in \mathcal{E}, h_e = h_u\}$; Time window threshold in the k-th layer τ^k; Neighborhood sample max size n; Non-linearity σ; Time encoding dimension m

Output: Multi-Flow Graph Features $z_e, \forall e \in \mathcal{E}$

1 **// Phase 1: Sinusoidal Temporal Encoding**
2 $\Phi(t_e) \leftarrow \text{TimeEncoder}(t_e), \forall e \in \mathcal{E}$
3 $\tilde{\mathbf{h}}_e^{(0)} \leftarrow f_e, \forall e \in \mathcal{E}$
4 **for** $k = 1 \dots K$ **do**
5 **for** $e \in \mathcal{E}$ **do**
6 **// Phase 2: Time-Aware Neighbor Sampling**
7 $\Delta t_{u,e} \leftarrow \text{TimeLag}(\Phi(t_u), \Phi(t_e)), \forall u \in C(e)$
8 ▷ Derive relative interval from temporal vectors
9 $\mathcal{N}_t^{(k)}(e) \leftarrow \{\forall u \in C(e) \vee \Delta t_{u,e} \le \tau^k\}$
10 $\mathcal{N}_{\text{sorted}}^{(k)}(e) \leftarrow \text{Sort}(\mathcal{N}_t^{(k)}(e),\ \Delta t_{u,e},\ \text{ascending})$
11 $\mathcal{N}_{\text{sampled}}^{(k)}(e) \leftarrow \text{TOP}_n\left(\mathcal{N}_{\text{sorted}}^{(k)}(e)\right)$
12 **// Phase 3: Time-Weighted Aggregation**
13 $w_{u,e} = \exp(-\lambda \Delta t_{u,e}), \forall u \in \mathcal{N}_{\text{sampled}}^{(k)}(e)$
14 ▷ Calculate temporal decay weights
15 $\mathbf{h}_{\mathcal{N}(e)}^{(k)} \leftarrow \sigma\left(\sum_u \frac{w_{u,e}}{\sum w} \tilde{\mathbf{h}}_u^{(k-1)}\right)$
16 $\tilde{\mathbf{h}}_e^{(k)} \leftarrow \sigma\left(\text{CONCAT}(\mathbf{W}_{\text{self}}^{(k)} \tilde{\mathbf{h}}_e^{(k-1)}, \mathbf{h}_{\mathcal{N}(e)}^{(k)})\right)$
17 ▷ Update edge representation
18 $\tilde{\mathbf{h}}_e^{(k)} \leftarrow \tilde{\mathbf{h}}_e^{(k)} / ||\tilde{\mathbf{h}}_e^{(k)}||_2, \forall e \in \mathcal{E}$
19 $\mathbf{z}_e^K \leftarrow \tilde{\mathbf{h}}_e^{(K)}, \forall e \in \mathcal{E}$
20 **for** $e \in \mathcal{E}$ **do**
21 $\mathbf{z}_e^K \leftarrow \text{CONCAT}(f_e, \mathbf{z}_e^K)$ ▷ Final feature extraction

a) Sinusoidal Temporal Encoding (Algorithm 2, Line 2). To model the sequential dependencies of concurrent flows without introducing absolute timestamp bias, we project flow initiation timestamps into a continuous vector space. Inspired by Transformer positional encodings [31], we apply a sinusoidal mapping $\Phi(t_e)$ as shown in Eq. (6).

$$\Phi(t_e)_{2i} = \sin\left(\frac{t_e}{\tau_{base}^{2i/d_p}}\right), \quad \Phi(t_e)_{2i+1} = \cos\left(\frac{t_e}{\tau_{base}^{2i/d_p}}\right) \tag{6}$$

Note that d_p is the encoding dimension and $\tau_{base} = 10000$. Instead of directly concatenating $\Phi(t_e)$ with node features, we maintain it as a parallel structural context to calculate the relative time lag $\Delta t_{u,e}$ between flows, dynamically modulating subsequent neighbor filtering and aggregation.

b) Time-Aware Neighbor Sampling (Algorithm 2, Lines 7-11). Malware attacks typically exhibit strong temporal locality [20], while older flows often represent unrelated background noise. To maintain computational efficiency and focus on highly correlated events, we dynamically filter the host-centric neighborhood C_e. At each layer k, we restrict candidate neighbors to a sliding time window τ^k and retain only the top-n most recent interactions:

$$\mathcal{N}_t^{(k)}(e) = \text{Top-}n\left\{u \in C_e \mid \Delta t_{u,e} \le \tau^k\right\} \tag{7}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

c) Time-Weighted Aggregation (Algorithm 2, Lines 13-21). Standard graph convolutions treat all neighbors equally, failing to capture the time-sensitive nature of attack behaviors. We introduce a time decay factor λ to assign higher attention weights to recent interactions. For each neighbor u, the weight $w_{u,e}$ is computed as shown in Eq. (8).

$$w_{u,e} = \exp(-\lambda \cdot \Delta t_{u,e}) \tag{8}$$

A positive λ ensures recent edges receive higher weights ($w_{u,e} \approx 1$ when $\Delta t \to 0$). Neighbor features are then aggregated into a latent space as detailed in Eq. (9):

$$\mathbf{h}_{\mathcal{N}(e)}^{(k)} = \sigma\left(\sum_{u \in \mathcal{N}_{\text{sampled}}^{(k)}} \frac{w_{u,e}}{\sum w} \tilde{\mathbf{h}}_u^{(k-1)}\right) \tag{9}$$

where $\frac{w_{u,e}}{\sum w}$ is the normalized temporal coefficient ensuring scale-invariance, and $\tilde{\mathbf{h}}_u^{(k-1)}$ denotes the neighbor representation from the previous iteration (with $\tilde{\mathbf{h}}^{(0)} = \mathcal{F}$).

To update the edge representation, we concatenate self-information with aggregated context, as shown in Eq. (10).

$$\tilde{\mathbf{h}}_e^{(k)} = \sigma\left(\text{CONCAT}(\mathbf{W}_{\text{self}}^{(k)} \tilde{\mathbf{h}}_e^{(k-1)}, \mathbf{h}_{\mathcal{N}(e)}^{(k)})\right) \tag{10}$$

Here, $\mathbf{W}_{\text{self}}^{(k)}$ is a learnable matrix designed to preserve the edge's own information from the previous state, while $\sigma(\cdot)$ (ReLU) mitigates gradient vanishing. After K iterations, the representation is normalized to a unit vector (Algorithm 2, line 18). For notational convenience, we denote the final representation output at depth K as $\mathbf{z}_e^K$ (Algorithm 2, line 19). Finally, to prevent the dilution of original single-flow features f_e, we extract *MGF* (i.e., $\mathbf{z}_e^K$) as shown in Eq. (11).

$$\mathbf{z}_e^K = CONCAT(f_e, \mathbf{z}_e^K) \tag{11}$$

C. Early-Stage Malware Detection

In this module, we propose a ViT-based detector to capture both local and global contextual relationships and achieve early-stage encrypted malware traffic detection, consisting of an input layer and a ViT-based architecture, as follows:

1) Input Layer: The proposed ViT-based detector accepts the output *MGF* (i.e., $\mathbf{z}_e^K$) of the multi-flow feature extraction as input, represented as a 2D matrix $\mathbb{M} \in \mathbb{R}^{2\times(Q+L)}$ with a single channel ($1 \times 2 \times (Q + L)$). By adjusting the number of packets M, the number of bytes for the packet header X, and the payload Y, we extract various lengths of *PBM*, i.e., Q. By adjusting the output length of the multi-flow feature embedding, we extract various lengths of $\mathbf{z}_e^K$, i.e., L.

2) ViT-Based Architecture: We adopt the ViT framework [32] to capture both local and global contextual relationships in the traffic data. The architecture consists of three main components as follows.

a) Patch Embedding and Positional Encoding. We split the input tensor into non-overlapping patches. Each patch is projected into an embedding space through a linear layer. To retain positional information, we add learnable 1D positional embeddings and a `[CLS]` token to aggregate global features.

b) Transformer Encoder. The embedded patches pass through six Transformer encoder layers, each consisting of Multi-Head Self-Attention (MHSA) with eight heads to model long-range dependencies, layer normalization after each sub-layer, a position-wise feed-forward network with Gaussian Error Linear Unit (GELU) activation, and residual connections around each sub-layer. Specifically, MHSA projects the queries, keys, and values into h independent subspaces through linear transformations in Eq. (12).

$$\begin{aligned} &MHSA(Q, K, V) \\ &= CONCAT(head_1, head_2, \ldots, head_h)W^O \end{aligned} \tag{12}$$

Note that $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$, and W_i^Q, W_i^K, W_i^V, and W^O are all learnable parameter matrices.

Following the self-attention mechanism in our ViT architecture, the second sublayer implements a position-wise feed-forward network (FFN), which consists of two consecutive linear transformations with a GELU activation in Eq. (13).

$$FFN(x) = W_2 \cdot GELU(W_1 x + b_1) + b_2 \tag{13}$$

Here, W_1 and W_2 are the weight parameters, b_1 and b_2 are the bias parameters.

c) Classification Head. We apply a lightweight classification head to the final `[CLS]` token embedding, consisting of layer normalization and a linear layer with a softmax function. The model is optimized via Adam with weight decay and cross-entropy loss, using dropout for regularization.

V. Theoretical Analysis

To quantitatively analyze the optimality of *DawnGuard*, we model the malware traffic detection process as an information preservation problem. Let the optimal detection performance be bounded by the mutual information $I(\mathcal{X}; Y)$ between the raw traffic space $\mathcal{X}$ and the label space Y. We demonstrate that our three-stage design minimizes the information loss $\mathcal{L}_{info}$ compared to existing methods.

A. Entropy Model Setup

We formalize a single flow as a random variable sequence $X = \{x_1, x_2, \ldots, x_L\}$ governed by a distribution $P(X)$. For detection methods treating flows in isolation, performance is theoretically bounded by the mutual information $I(X; Y)$ with the label Y. By the data processing inequality [33], any feature extraction $\phi(X)$ yields $I(\phi(X); Y) \leq I(X; Y)$, establishing an information barrier for intra-flow methods.

However, in real-world networks, X correlates with neighbor flows, denoted as context $\mathcal{C}$. Consequently, the true approachable bound is their joint mutual information: $I(X, \mathcal{C}; Y) = I(X; Y) + I(\mathcal{C}; Y|X)$. While single-flow baselines are limited to $I(X; Y)$, our graph-based framework leverages this expanded observable space to capture the conditional mutual information $I(\mathcal{C}; Y|X)$, fundamentally raising the theoretical upper bound of detection accuracy.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

B. Analysis of Main Modules in DawnGuard

1) Self-Adjusting Augmentation as Optimal Information Selection: Unlike simply assigning more weight to early stages of data, our Self-Adjusting Data Augmentation identifies and preserves the region of highest information density. Given a flow X with packet densities $D = \{d_1, \ldots, d_L\}$, the goal of tail masking is to find an optimal truncation point k^* such that the retained subsequence $X_{1:k^*}$ maximizes the relevant information density while minimizing redundancy, as shown in Eq. (14).

$$k^* = \arg\max_k \frac{I(X_{1:k}; Y)}{k} \quad \text{s.t.} \quad k \geq \alpha L \tag{14}$$

Our dual-threshold mechanism (Algorithm 1) dynamically estimates this k^* by detecting the plunge point of packet density. By synthesizing augmented samples $\{X^{(i)}_{1:k^*}\}$ concentrated around this high-density phase (e.g., handshakes), we reconstruct a training distribution $\hat{P}_{aug}$ that approximates the true intrinsic distribution P_{int} of malware behaviors more accurately than the raw distribution P_{raw}. Thus,

$$H(\hat{P}_{aug}) \approx H(P_{int}) \implies I(D_{aug}; Y) \gg I(D_{static}; Y) \tag{15}$$

This proves that our density-aware strategy theoretically retains more discriminative information than static weighting.

2) Temporal Graph Learning as Causal Information Recovery: While existing graph learning based methods exploit contextual information, they typically treat all neighbors $\mathcal{N}(e)$ equally, ignoring the temporal interdependencies between multiple flows. In malware detection, attack patterns exhibit strong causal dependencies. Treating historical neighbors (high Δt) as equivalent to recent ones introduces *temporal noise*, which increases the conditional entropy $H(Y|\mathcal{N}(e))$. Our module addresses this by maximizing the Causal Mutual Information through two mechanisms as follows.

a) Time-Aware Sampling as Relevance Denoising: Let the neighborhood set $\mathcal{N}_{all}$ contain all historical interactions. Most elements in $\mathcal{N}_{all}$ are statistically independent of the current attack state Y due to large time lags (i.e., larger than 5 minutes [20]). *DawnGuard* filters neighbors using a sliding window τ^k to construct a subset $\mathcal{N}^*_t$. This process can be modeled as searching for the optimal context window that maximizes the information density, as shown in Eq. (16).

$$\mathcal{N}^*_t = \arg\max_{\mathcal{S} \subseteq \mathcal{N}_{all}} \frac{I(\mathcal{S}; Y)}{|\mathcal{S}|} \quad \text{s.t.} \quad \forall u \in \mathcal{S}, \Delta t_{u,e} \leq \tau^k \tag{16}$$

By discarding outdated interactions, we effectively prune the graph topology to remove uncorrelated edges that would act as entropy noise sources.

b) Time-Weighted Aggregation as Causal Weighting: Even within the active window, relevance is different. The standard aggregation sum $\sum h_u$ assumes a uniform prior on neighbor importance [27]. Our decay mechanism $w_{u,e} = \exp(-\lambda \Delta t_{u,e})$ introduces a temporal prior $P_{time}(\Delta t)$ that aligns with the natural decay of network causality. The aggregated representation $\mathbf{h}^{(k)}_{\mathcal{N}(e)}$ thus represents a weighted information integration:

$$I(\mathbf{h}^{(k)}_{\mathcal{N}(e)}; Y) \approx \sum_{u \in \mathcal{N}^*_t} w_{u,e} \cdot I(u; Y) \tag{17}$$

Specifically, our mechanism aligns the attention weights $w_{u,e}$ with the temporal decay of causal dependencies, thereby prioritizing recent neighbors that inherently maximize the mutual information $I(u; Y)$. This targeted prioritization allows the model to effectively extract temporal discriminative patterns while suppressing interference from outdated noise.

3) ViT as Optimal Information Aggregation: Finally, we analyze the encoding efficiency. Let Z be the final representation. A standard CNN encoder with local receptive fields effectively applies a mask M, losing long-range dependencies: $I(Z_{CNN}; Y) = I(X; Y) - H(X_{long})$. In contrast, the ViT architecture in *DawnGuard*, with its global self-attention mechanism, maintains a global receptive field. Thus, it minimizes the encoding loss, as shown in Eq. (18).

$$\mathcal{H}_{DawnGuard} \geq \mathcal{H}_{CNN} \implies I(Z_{ViT}; Y) \approx I(Z_{optimal}; Y) \tag{18}$$

C. Quantitative Analysis of Information Loss

To evaluate the theoretical optimality discussed in Section V-B, we conduct a quantitative evaluation of the information preservation capabilities of *DawnGuard*. Following the entropy model setup, we need to measure the Mutual Information $I(Z; Y)$ between the extracted high-dimensional continuous features Z and the discrete label space Y. To quantify this, we utilize the Kraskov-Stögbauer-Grassberger (KSG) estimator [34]. Because traditional binning-based methods suffer from severe bias in high-dimensional continuous spaces. In contrast, the KSG estimator provides estimation based on k-nearest neighbors. It adjusts neighborhood distances to mitigate estimation bias for continuous, discrete variable pairs without requiring prior assumptions about the data distribution, thereby ensuring the reliability of our information loss quantification. Our evaluation subset comprises randomly selected 1000 TrickBot samples in the CTU-Malware dataset [19] from D_1 and 1000 benign samples from D_4.

As shown in Table III, the results demonstrate a significant reduction in information loss across all three dimensions:

1) Sparsity-Induced Loss Reduction: Compared to applying a static high weight to the first 30% traffic, our self-adjusting data augmentation preserves more discriminative information by dynamically locating high-density interaction phases.

2) Independence-Assumption Loss Reduction: By expanding the observation space from isolated single flows to temporal multi-flow graphs, *DawnGuard* captures the conditional mutual information $I(\mathcal{C}; Y|X)$, and reduces the entropy noise from irrelevant background traffic.

3) Locality-Induced Loss Reduction: The ViT-based architecture, benefiting from its global self-attention mechanism, minimizes encoding loss by maintaining long-range dependencies that are often masked by the local receptive fields of CNN-based models.

These findings demonstrate that unlike the methods that suffer from sparsity-induced loss (low $\hat{H}$), independence-assumption loss (missing $I(\mathcal{C}|X)$), and locality-induced loss (masking X_{long}), *DawnGuard* minimizes these information losses, theoretically approaching the detection optimality.

VI. Performance Evaluation

We prototype *DawnGuard* and conduct extensive experiments to evaluate its effectiveness. Moreover, we compare its accuracy with state-of-the-art methods on existing datasets, including two well-known real-world datasets. In general, the experiments will demonstrate that *DawnGuard* is able to:

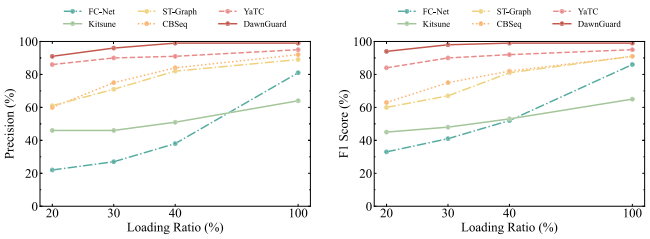
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE III
QUANTITATIVE REDUCTION OF INFORMATION LOSSES ACROSS DIFFERENT MODULES IN DAWNGUARD

Loss Category	Baseline Method	*DawnGuard* Module	$L_{baseline}$ (bits)	$L_{DawnGuard}$ (bits)	Reduction (ΔL)
Sparsity-induced	Static Weighting	Self-Adjusting Data Augmentation	0.994	0.650	0.343 ↓
Independence-assumption	Single-flow PBM	Multi-flow Temporal Graph Learning	0.650	0.518	0.132 ↓
Locality-induced	CNN-based ResNet50	Early-stage Malware Detection	0.604	0.454	0.150 ↓

[1] The initial entropy $H(Y)$ is normalized to 1.0 bit. L denotes the information loss calculated by $H(Y) - I(Z;Y)$ using the KSG estimator.

- detect various malware traffic in varying early-stage loading ratios and time windows (Section VI-B).
- achieve reliable early-stage malware detection in highly imbalanced data scenarios, effectively mitigating the bias towards the majority benign class (Section VI-C).
- achieve zero-shot detection ability to capture unseen patterns of malware traffic generated by unknown malware types (Section VI-D).
- exhibit strong stability across different parameter configurations (Section VI-E).



(a) Pre at different loading ratios (b) F1 at different loading ratios

Fig. 5. The evaluation results (%) of different detection methods at different loading ratios of malware in D_3.

A. Experiment Setting

Baseline Methods. To measure the improvement achieved by *DawnGuard*, we establish five baselines, among which two are multi-flow-based, i.e., ST-Graph [2] and CBSeq [3]. Additionally, YaTC [16] is included as a representative baseline that enables detection without requiring complete traffic.

- **Kitsune** [13] uses unsupervised autoencoders for feature learning and anomaly detection.
- **FC-Net** [15] detects malicious traffic by extracting image-based feature maps from raw flows.
- **ST-Graph** [2] explores network behaviors using graph learning frameworks integrated with TLS metadata.
- **CBSeq** [3] characterizes attacking intents via a Transformer-based multi-flow representation.
- **YaTC** [16] leverages a Masked Autoencoder architecture and raw-byte representations for traffic classification.

Evaluation Dataset. As shown in TABLE IV, we use the following public datasets for evaluation.

- **CTU-Malware** [19]. Comprising continuous real-world PCAP traces, this dataset preserves flow chronology for the temporal-topological reconstruction of multi-flow attacks. We select six malware families (Dridex, Emotet, Geodo, Miuref, Zeus, TrickBot) for D_1, reserving the remainder (D_2) for unseen malware evaluation.
- **USTC-TFC2016** [35]. Comprising ten categories each of benign and malware traffic, this dataset's timestamps and IPs are used to establish session contexts, grouping interactions into local subgraphs. D_3 and D_4 are constructed from its malware and benign subsets, respectively.

Evaluation criteria. The fundamental goal of encrypted malware traffic detection is to be accurate. To achieve this goal, we mainly use the F1 and AUC because they are most widely used in the literature [3], [4], [13]. We also use precision to validate *DawnGuard*'s improvements. The formal definitions of these metrics are presented in Eq. 19 and 20.

$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN} \tag{19}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{AUC} = \int_0^1 ROC(x)\,dx \tag{20}$$

TP, TN, FP, and FN denote the number of True Positives, True Negatives, False Positives, and False Negatives.

Implementation Details. To ensure the reproducibility of our results and facilitate fair comparisons with baselines, we provide the implementation details as follows.

- **Setup & Scalability.** All experiments were conducted on a server with an Intel Core i73.4 GHz, 32GB RAM, and an RTX 3080 using PyTorch. To ensure scalability on RAM-constrained hardware, *DawnGuard* avoids full dataset loading by streaming traffic via mini-batches.
- **Data Partitioning & Preprocessing.** Datasets (D_1–D_4) were split into training, validation, and testing sets at flow granularity via stratified sampling with an 8 : 1 : 1 ratio. To build inter-flow interaction graphs, we sorted flows chronologically, which bridges the gap between discrete flows and the required graph structure.
- **Evaluation Setting.** For fair comparison, baseline hyperparameters were tuned on our validation set to match or exceed their reported performance. Under loading ratio η, input traffic was truncated for all methods to the same initial segment and all statistics were derived from these observed segments to eliminate look-ahead bias.
- **Validation.** It is important to note that the datasets used in training and testing are entirely independent. To reduce the impact of randomness in sample selection, we repeat all experiments ten times to ensure more credible results.
- **Hyperparameter Settings.** *Data Augmentation:* retention ratio $\alpha = 0.1$, min packets $\mu = 6$, Savitzky-Golay smoothing (window 11, poly_order 3), threshold coefficient $\gamma = 0.3$, peak density bound $\beta = 0.15$, and soft threshold angle $\theta = 30°$. *Feature Extraction:* initial packets $M = 6$, header bytes $X = 80$, and payload bytes $Y = 240$ [16]. *Graph Learning:* time window $\tau^k = 5$ minutes [20], max neighbors $n = 8$, depth $K = 2$, time encoding dimension

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

SUMMARY OF THE DATASETS WITH TRAFFIC TYPES AND FLOW COUNTS PER CATEGORY

Dataset	Dataset ID	Traffic Types
CTU-Malware [19]	D_1	Dridex (26046), Emotet (21580), Geodo (12252), Miuref (16008), Zeus (16898), TrickBot (24024)
	D_2	Unseen Malware [containing Cridex, Bunitu, Tinba, Ransomware, DownloadGuide, etc] (175002)
USTC-TFC2016 [35]	D_3	Cridex (16385), Geodo (40947), HTBot (6341), Miuref (13481), Neris (33890), Nsis-ay (6165), Shifu (9634), Tinba (8505), Virut (33147), Zeus (10972)
	D_4	Benign (309887)

TABLE V

THE EVALUATION RESULTS (%) OF DIFFERENT DETECTION METHODS AT DIFFERENT LOADING RATIOS OF MALWARE IN D_1

η	Malware	FC-Net [15]			Kitsune [13]			ST-Graph [2]			CBSeq [3]			YaTC [16]			*DawnGuard*		
		Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC
20%	Dridex	22.82	35.64	52.17	32.55	32.81	54.97	73.41	71.36	80.97	71.31	68.97	82.26	98.51	83.10	97.84	93.46	95.41	97.83
	Emotet	**21.87**[1]	33.43	56.72	34.90	35.47	56.68	71.27	67.74	77.93	68.47	72.10	84.62	90.91	85.80	96.72	94.26	95.97	97.96
	Geodo	25.59	37.21	59.75	10.71	–[2]	43.10	**62.88**	65.32	79.32	67.85	67.70	80.89	86.71	92.88	97.66	93.69	94.96	97.02
	Miuref	23.96	38.09	62.14	11.31	–	40.90	73.97	72.05	81.85	75.27	73.98	82.57	73.75	84.89	97.99	**90.33**	93.98	97.52
	Zeus	27.33	35.20	56.71	**10.26**	–	24.62	64.83	65.97	77.36	74.16	73.96	82.83	96.13	80.12	97.13	94.01	95.16	97.31
	TrickBot	26.96	38.36	61.55	17.82	–	42.50	66.30	63.41	78.96	**56.04**	64.31	80.61	**69.14**	81.75	97.25	93.80	95.47	97.92
	Average	24.76	36.78	58.17	19.59	–	43.80	68.78	67.71	79.40	68.85	70.42	82.30	85.86	86.41	97.43	**93.22**	**95.11**	**98.59**
30%	Dridex	69.56	77.59	89.33	42.94	36.71	58.85	80.67	80.06	88.58	79.79	79.17	87.59	97.52	85.30	95.67	96.65	96.44	98.86
	Emotet	**35.87**	51.37	76.65	40.32	38.03	59.01	84.10	81.16	86.42	84.70	78.54	86.33	92.41	92.06	98.61	96.26	96.37	98.91
	Geodo	69.56	72.10	84.22	18.90	–	44.65	**79.08**	78.89	86.79	**73.73**	77.44	87.35	90.25	94.87	98.98	**95.10**	95.89	98.77
	Miuref	43.43	53.31	73.66	11.37	–	30.27	80.79	82.77	89.11	78.75	75.22	82.57	**86.08**	92.05	98.35	96.92	97.17	99.27
	Zeus	54.45	69.13	87.74	**10.14**	–	23.86	84.12	86.45	91.69	77.72	78.86	87.65	93.14	90.82	98.42	95.41	96.08	98.62
	TrickBot	52.87	62.60	79.97	19.94	–	43.67	79.48	81.00	88.06	75.91	73.22	81.37	87.99	93.61	98.98	96.10	96.66	99.14
	Average	54.29	65.40	81.93	23.94	–	43.39	81.37	81.76	88.44	78.43	77.20	85.48	91.23	91.86	98.17	**96.07**	**96.44**	**98.93**
40%	Dridex	59.57	68.64	89.01	54.64	37.54	61.25	90.24	91.32	94.81	94.34	87.00	89.04	**87.22**	93.17	98.92	97.31	97.77	99.85
	Emotet	67.14	68.78	92.25	44.29	39.32	60.19	91.28	89.37	91.98	89.04	85.74	90.45	93.90	96.54	98.73	97.07	97.62	99.74
	Geodo	72.85	70.21	85.60	20.19	–	44.38	88.66	88.84	92.37	**86.47**	84.18	90.50	94.02	96.92	98.94	97.03	97.26	99.29
	Miuref	62.23	60.31	81.62	18.29	–	38.22	86.64	87.60	92.82	88.40	84.36	88.13	92.70	96.21	98.99	96.53	96.82	99.27
	Zeus	**59.56**	68.88	80.59	**12.20**	–	27.00	89.49	89.75	93.96	89.34	87.49	91.65	92.03	94.76	98.51	**96.25**	96.69	99.21
	TrickBot	72.93	69.29	79.31	20.34	–	43.50	**85.69**	85.51	90.89	92.93	86.36	91.11	90.45	94.61	98.55	96.49	97.46	99.76
	Average	68.05	67.69	84.73	28.33	–	45.76	88.67	88.74	92.81	90.09	85.89	90.15	91.72	95.39	98.77	**96.78**	**97.27**	**99.52**
100%	Dridex	80.87	89.42	96.65	**20.05**	–	40.35	98.98	97.02	97.43	94.21	97.02	99.34	99.39	99.56	99.79	99.64	99.68	99.92
	Emotet	80.63	89.28	96.88	20.06	33.42	56.51	98.94	97.19	97.55	95.68	97.79	99.47	99.73	99.86	99.97	98.94	99.32	99.82
	Geodo	**77.50**	87.32	96.09	20.08	–	44.76	97.58	95.98	96.59	**88.66**	91.32	95.43	99.31	99.65	99.91	98.97	99.31	99.79
	Miuref	80.13	88.99	96.63	20.75	–	45.96	99.24	99.15	99.39	90.84	93.02	96.47	99.94	99.64	99.61	98.67	99.11	99.71
	Zeus	76.25	86.52	95.86	21.00	–	47.89	**96.60**	96.51	97.95	90.14	94.81	98.66	99.88	99.05	99.18	99.55	99.63	99.91
	TrickBot	79.00	88.26	97.03	**20.05**	–	40.76	97.58	95.98	96.68	96.10	98.01	99.43	**95.11**	97.49	99.44	**98.57**	99.09	99.73
	Average	79.06	88.30	96.52	20.33	–	46.04	98.15	96.97	97.60	92.61	95.33	98.13	98.89	99.21	99.65	**99.06**	**99.36**	**99.81**

[1] We make the minimum precision bold and underline it among the six malware to evaluate the reliability of detection methods on early-stage traffic.
[2] We mark – for the F1 when the AUC is lower than 0.50, which is the accuracy of random guessing.

TABLE VI

THE EVALUATION RESULTS (%) OF DIFFERENT DETECTION METHODS AT DIFFERENT EARLY-STAGE TIME WINDOWS OF MALWARE IN D_1

t	FC-Net [15]			Kitsune [13]			ST-Graph [2]			CBSeq [3]			YaTC [16]			*DawnGuard*		
	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC
1s	80.32	48.09	66.88	20.07	–	48.24	77.74	72.45	81.56	19.73	–	39.70	55.60	70.97	88.62	**96.21**	**96.48**	**98.67**
3s	83.40	63.24	74.90	20.53	32.94	50.43	81.15	79.00	86.14	54.36	52.92	70.36	61.66	75.67	91.29	**97.71**	**97.96**	**98.82**
5s	78.96	73.16	81.76	20.49	–	49.30	82.93	82.87	88.76	48.03	49.13	68.26	73.73	83.26	93.76	**98.13**	**98.24**	**99.72**
10s	81.91	87.64	94.49	20.81	–	48.56	89.15	87.66	91.63	92.09	83.41	87.56	85.42	91.06	97.75	**98.79**	**98.84**	**99.88**

[1] We mark – for the F1 when the AUC is lower than 0.50, which is the accuracy of random guessing.

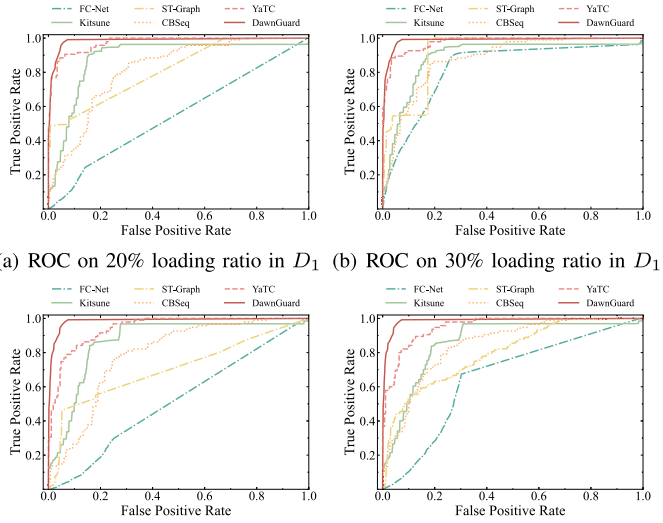
$m = 64$, edge embedding dimension $\omega = 512$, and dropout 0.1. *Detection Phase:* 2D matrix $\mathbb{M} \in \mathbb{R}^{2\times 1088}$ (fusing 1920 single-flow and 256 multi-flow features), patch size 2×64. The Transformer Encoder uses 6 layers, 8 attention heads, and a hidden dimension of 512. We trained the model with a batch size of 150, learning rate 0.0001, and weight decay 0.001.

B. Malware Detection in Varying Early-Stage Time Window

We evaluate detection performance in relatively early-stage windows (loading ratios $\eta \in \{20\%, 30\%, 40\%, 100\%\}$) and absolute windows ($t \in \{1s, 3s, 5s, 10s\}$) based on Table I. We build binary classifiers for malware in D_1, D_3, and benign traffic in D_4. To observe the performance of these methods in detecting early-stage malware traffic, we generate testing traffic in different loading ratios and time windows based on packet timestamps. The results are illustrated in Fig. 5 and 6, TABLE V and VI.

As shown in TABLE V and Fig. 5, itDawnGuard achieves the best detection F1 and AUC under each early-stage time window setting in both evaluation datasets. Specifically, Fig. 5 (a) and (b) visualize the Precision and F1 trends on D_3. It is evident that while the performance of all methods improves

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.



(a) ROC on 20% loading ratio in D_1 (b) ROC on 30% loading ratio in D_1

(c) ROC on 20% loading ratio in D_3 (d) ROC on 30% loading ratio in D_3

Fig. 6. ROC of different detection methods at different loading ratios of malware in D_1 and D_3.

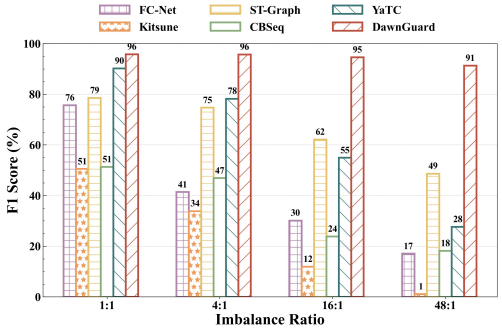
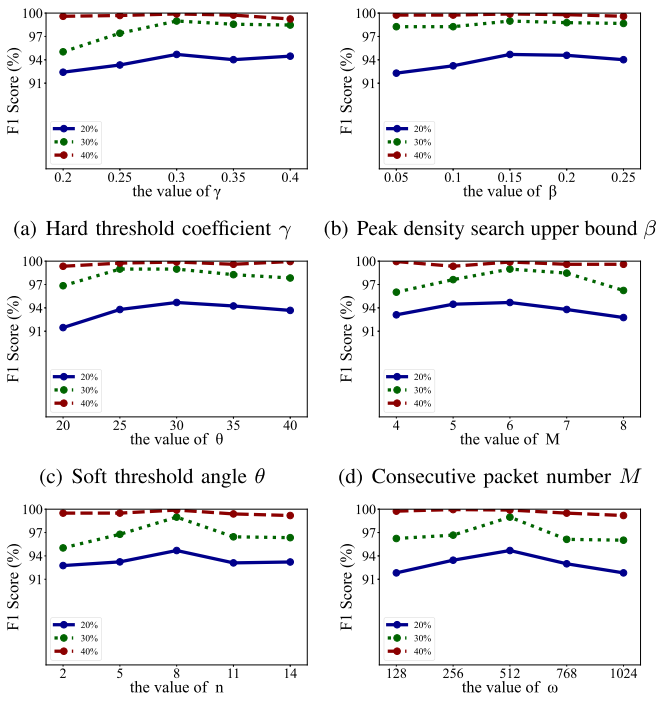


Fig. 7. The early-stage unseen malware detection F1 of different methods under different imbalance ratios in D_2.

as the loading ratio increases from 20% to 100%, *DawnGuard* maintains a consistently high performance curve. In contrast, the curves for baseline methods like FC-Net and Kitsune show a sharp decline in the low loading ratio region (e.g., 20%-30%), visually demonstrating their inability to handle sparse early-stage information. Even in the smallest loading ratio scenario in D_1, i.e., $\eta = 20\%$, where data is most sparse, *DawnGuard* still achieves an average F1 of 95.11%, which is 8.7% improvement over the best accuracy of the SOTA methods. *DawnGuard* demonstrates a significant advantage in early-stage detection compared with the SOTA methods, which can be primarily attributed to our self-adjusting data augmentation and the MGF representation. The self-adjusting data augmentation dynamically creates effective training samples from limited early-stage data (see the 22.33% F1 drop without the data augmentation module in ablation study in Fig. 9), while the MGF representation's ability to model inter-flow dependencies allows it to extract meaningful patterns even from scarce data (see the 14.57% F1 drop without multi-flow features in ablation study in Fig. 9). In contrast, methods relying on single flow or static graph fail to compensate for the information scarcity. The results at different specific-size time windows are illustrated in TABLE VI and also demonstrate the effectiveness of *DawnGuard* in early-stage detection.



(a) Hard threshold coefficient γ (b) Peak density search upper bound β

(c) Soft threshold angle θ (d) Consecutive packet number M

(e) Neighborhood sample max size n (f) Edge embedding dimension ω

Fig. 8. The detection F1 (%) of different parameter settings at different loading ratios of malware in D_3.

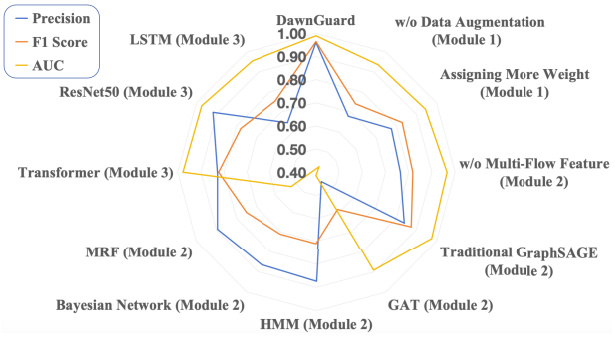


Fig. 9. Ablation study: comparing existing models.

As shown in TABLE V, *DawnGuard* consistently outperforms baselines across all early-stage settings, exhibiting the highest average and minimum precision on the six malware families. This demonstrates its reliability in early-stage detection. It stems from our multi-flow temporal graph learning, which extracts distinguishable properties among multiple flows to precisely detect diverse malware attacks.

In addition, we also evaluate the performance of *DawnGuard* under the complete dataset without splitting early-stage time windows. As shown in TABLE V and Fig. 5, it can be found that *DawnGuard* achieves the F1 of 99.36% and AUC of 99.81% on average in D_1, and the F1 of 99.88% and AUC of 99.99% on average in D_3, which are higher than the best accuracy of the SOTA methods. While Kitsune fails in D_1 due to ineffective feature extraction, these results confirm that our *MGF* representation efficiently captures discriminative features for malware traffic detection.

To further evaluate the robustness against false positives, which is critical for deployment in large-scale networks, we analyze the Receiver Operating Characteristic (ROC) curves of

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

THE EVALUATION RESULTS (%) OF DIFFERENT DETECTION METHODS UNDER DIFFERENT IMBALANCE RATIOS OF BENIGN AND MALWARE IN D_1

ρ	Malware	FC-Net [15]			Kitsune [13]			ST-Graph [2]			CBSeq [3]			YaTC [16]			*DawnGuard*		
		Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC
4:1	Dridex	69.56	77.59	89.33	42.94	36.71	58.85	80.67	80.06	88.58	79.79	79.17	87.59	97.52	85.30	95.67	96.65	96.44	98.86
	Emotet	**35.87**	51.37	76.65	40.32	38.03	59.01	84.10	81.16	86.42	84.70	78.54	86.33	92.41	92.06	98.61	96.26	96.37	98.91
	Geodo	69.56	72.10	84.22	18.90	–	44.65	**79.08**	78.89	86.79	**73.73**	77.44	87.35	90.25	94.87	98.98	**95.10**	95.89	98.77
	Miuref	43.43	53.31	73.66	11.37	–	30.27	80.79	82.77	89.11	78.75	75.22	82.57	**86.08**	92.05	98.35	96.92	97.17	99.27
	Zeus	54.45	69.13	87.74	**10.14**	–	23.86	84.12	86.45	91.69	77.72	78.86	87.65	93.14	90.82	98.42	95.41	96.08	98.62
	TrickBot	52.87	62.60	79.97	19.94	–	43.67	79.48	81.00	88.06	75.91	73.22	81.37	87.99	93.61	98.98	96.10	96.66	99.14
	Average	54.29	65.40	81.93	23.94	–	43.39	81.37	81.76	88.44	78.43	77.20	85.48	91.23	91.86	98.17	**96.07**	**96.44**	**98.93**
16:1	Dridex	62.35	64.24	79.94	6.74	12.41	53.85	**75.00**	68.78	82.37	47.35	48.31	68.01	26.65	42.07	90.84	97.31	97.42	98.62
	Emotet	62.26	64.08	80.65	6.60	12.06	53.30	76.28	70.93	79.93	47.10	47.98	67.58	70.22	79.40	94.69	91.46	94.47	98.38
	Geodo	61.81	63.25	80.90	6.06	11.07	51.08	82.95	80.81	87.30	**47.08**	47.78	67.47	60.24	75.19	97.53	97.22	97.26	98.81
	Miuref	**60.30**	60.52	78.47	5.92	–	48.57	85.41	82.17	88.29	47.84	50.37	68.15	**22.16**	36.29	93.14	**90.91**	93.80	98.13
	Zeus	61.79	63.59	82.19	5.93	10.90	51.65	45.62	49.20	73.91	51.59	52.90	70.40	35.60	52.43	93.93	96.22	96.47	98.22
	TrickBot	62.09	63.75	83.14	**5.44**	–	45.30	87.50	85.36	89.95	47.21	47.46	67.96	41.38	57.30	92.34	93.02	95.04	97.62
	Average	61.77	63.24	80.88	6.12	11.22	50.63	75.46	72.88	83.63	48.03	49.13	68.26	42.71	57.11	93.75	**94.36**	**95.74**	**99.30**
48:1	Dridex	41.18	55.93	76.66	0.61	–	20.56	72.64	67.19	78.97	54.17	35.62	66.32	33.48	50.17	97.66	94.07	93.22	96.02
	Emotet	43.48	58.79	82.01	0.58	–	20.18	55.94	58.45	79.42	24.82	35.35	72.65	18.75	30.04	85.89	93.16	94.40	98.11
	Geodo	**30.17**	43.97	69.07	**0.49**	–	20.24	**46.98**	47.44	70.60	**16.44**	26.87	74.84	11.78	21.07	91.39	91.74	92.78	96.61
	Miuref	31.82	43.95	66.47	0.57	–	21.74	55.78	63.59	87.99	26.32	12.65	71.97	10.45	18.92	89.32	**89.73**	92.82	98.02
	Zeus	37.42	50.08	71.94	0.55	–	24.28	60.45	60.95	78.23	18.65	25.77	68.05	20.13	33.47	95.29	90.61	92.19	96.91
	TrickBot	42.70	56.65	78.29	0.54	–	22.78	48.38	48.64	75.58	39.58	36.89	60.09	**10.29**	18.66	90.36	89.84	91.98	97.04
	Average	37.80	51.67	74.07	0.56	–	21.63	56.70	57.71	78.47	30.00	28.86	68.99	17.48	28.72	91.65	**91.53**	**92.90**	**99.54**

[1] We make the minimum precision bold and underline it among the six malware to evaluate the reliability of detection methods on early-stage traffic.
[2] We mark – for the F1 when the AUC is lower than 0.50, which is the accuracy of random guessing.

TABLE VIII

THE EVALUATION RESULTS (%) OF DIFFERENT DETECTION METHODS UNDER DIFFERENT IMBALANCE RATIOS OF BENIGN AND MALWARE IN D_3

ρ	FC-Net [15]			Kitsune [13]			ST-Graph [2]			CBSeq [3]			YaTC [16]			*DawnGuard*		
	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC	Pre	F1	AUC
4:1	71.15	70.21	79.37	46.19	47.85	64.82	79.25	80.44	91.17	75.30	74.71	84.95	90.23	89.99	94.32	**96.46**	**98.17**	**99.44**
16:1	65.59	65.95	74.15	17.10	22.19	60.45	73.99	73.13	78.34	34.36	37.24	60.48	61.83	72.48	92.43	**96.38**	**98.03**	**99.98**
48:1	60.98	63.31	71.29	3.16	5.03	52.06	50.48	52.48	67.68	12.46	14.19	51.26	52.13	49.91	89.98	**89.29**	**94.34**	**99.99**

different methods. Fig. 6 illustrates the ROC curves for loading ratios of 20% and 30% on both D_1 and D_3. We observe that *DawnGuard* consistently maintains a better performance, with its curve significantly closer to the top-left corner compared to all baselines. Specifically, as shown in Fig. 6(a) and (c), even at a challenging 20% loading ratio, *DawnGuard* achieves a high True Positive Rate (TPR) while suppressing the False Positive Rate (FPR) to a near-zero level. In contrast, baseline methods like Kitsune and FC-Net exhibit a much slower rise in TPR, indicating they would trigger significantly more false alarms to reach the same detection sensitivity. This high resistance to false positives demonstrates that our multi-flow temporal graph learning can extract discriminative features that effectively distinguish coordinated malware activities from benign traffic.

C. Early-Stage Malware Detection in Imbalanced Data

To evaluate the ability against imbalanced data in early-stage detection, we construct a binary classifier for each type of malware traffic in D_1 and D_3 and benign traffic in D_4. Performance across varying imbalance ratios of benign and malware samples $\rho \in \{4:1, 16:1, 48:1\}$ is summarized in TABLES VII and VIII.

As shown in TABLE VII, *DawnGuard* consistently outperforms baselines across all imbalance ratios. Even under extreme imbalance, i.e., $\rho = 48:1$, it achieves an average F1 of 92.9% and an AUC of 99.54%, surpassing the best SOTA method by 35.19% and 7.89%, respectively. This advantage stems from our self-adjusting data augmentation, which reinforces discriminative features in sparse early stages to mitigate bias towards the majority class. The evaluation results under different imbalanced ratios in D_3 are shown in TABLE VIII, which also demonstrates the generalization capability of *DawnGuard* in imbalanced data.

In addition, TABLE VII also illustrates that *DawnGuard* achieves the most stable detection performance in average and minimum precision across different malware under each imbalance ratio setting. From $\rho = 4:1$ enlarges to $\rho = 48:1$, average precision remains higher than 89% and only reduces by 4.54%, which is superior to the SOTA methods. In the highest imbalance ratio, i.e., $\rho = 48:1$, *DawnGuard* achieves an average and minimum precision of 91.53% and 89.73%, which are 34.83% and 42.75% higher than the SOTA method in this scenario, FC-Net.

D. Unseen Malware Early Detection in Imbalanced Data

In this section, we evaluate the ability of different methods to deal with imbalanced datasets in early-stage unseen malware detection. Specifically, we construct a binary classifier trained by D_1 and detect malware traffic in D_2 in the early stage. To observe the detection performance of these methods when the imbalance ratio changes, we set the imbalance ratios of the data as $\rho \in \{4:1, 16:1, 48:1\}$.

As shown in Fig. 7, itDawnGuard achieves the best unseen malware detection accuracy under each imbalance ratio setting. Even in the most highly imbalanced dataset scenario, i.e., $\rho = 48:1$, *DawnGuard* still achieves an average F1 of

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
EVALUATION OF EFFICIENCY: TRAINING AND TESTING TIME

Methods	Training Time (s)				Testing Time (ms)		
	FET	PTT	CTT	Total	FET	Detection Time	Total
FC-Net [15]	11.38	236.88	18.62	266.88	0.64	0.21	0.85
Kitsune [13]	24.52	–	25.52	50.06	3.66	0.08	3.74
ST-Graph [2]	51.03	–	44.62	95.65	3.93	0.46	4.39
CBSeq [3]	686.08	–	132.87	818.95	37.06	3.22	40.28
YaTC [16]	18.33	2105	35.8	2159.13	1.65	0.52	2.17
DawnGuard	118.5	–	32.47	150.97	1.29	0.21	1.5

TABLE X
EUCLIDEAN DISTANCE BETWEEN MALWARE AND BENIGN TRAFFIC UNDER DIFFERENT REPRESENTATIONS

	ST-Graph [2]	Color Image [15]	MFR [16]	*MGF*
Dridex	0.27	0.52	0.53	**0.60**
Emotet	0.29	0.48	0.47	**0.60**
Miuref	0.27	0.41	0.44	**0.59**
Geodo	0.27	0.41	0.45	**0.61**
Zeus	0.28	0.43	0.43	**0.56**
TrickBot	0.27	0.49	0.47	**0.61**
Average	0.28	0.46	0.47	**0.60**

91.34%, which is 42.68% higher than the SOTA method in this scenario, i.e., ST-Graph, demonstrating *DawnGuard* can extract general features of malware traffic and remain resistant in the high imbalance ratio. When the imbalance ratio enlarges, FC-Net, CBSeq, and YaTC lose the ability to detect unseen malware traffic, which achieves F1 of 17.08%, 18.18%, and 27.63%, respectively, when $\rho = 48:1$. This is because the features extracted by these methods are not distinguishable enough to detect unseen malware traffic, evidenced by quantitative analysis of feature representation in TABLE X.

E. Parameter Sensitivity Analysis

We further study the impact of different parameter values on the performance of *DawnGuard*. We select six key parameters across different modules, including the hard threshold coefficient γ, the peak density search upper bound β, the soft threshold angle θ, the number of initial consecutive packets M, the neighborhood sample max size n, and the dimension of edge embeddings ω. We measure the F1 of *DawnGuard* in D_3 under different loading ratios (i.e., 20%, 30%, and 40%).

As shown in Fig. 8, itDawnGuard demonstrates strong robustness to these parameters. For instance, the number of initial packets M, which defines the input granularity of single-flow features, shows minimal impact on the F1 within the range of $[4, 8]$. Specifically, at a 20% loading ratio, the F1 only varies between 92.76% and 94.69% as M changes. Similarly, the performance remains stable when the edge embedding dimension ω varies from 128 to 1024, and when the neighborhood sample size n ranges from 2 to 14. The high stability across these parameters, particularly those governing the self-adjusting augmentation (i.e., γ, β, θ), confirms that *DawnGuard* effectively captures the intrinsic patterns of early-stage traffic through its design rather than relying on an over-optimized parameter space. In conclusion, the performance of *DawnGuard* is not sensitive to parameter choices.

F. Evaluation of Efficiency

We evaluate the time cost of all methods across training and testing phases, as summarized in TABLE IX. The training process comprises feature extraction time (FET), classifier training time (CTT), and pre-training time (PTT) specifically for meta-learning methods. Testing time represents the inference latency of the trained classifier. We set the sample size for model retraining to 10.

As shown in TABLE IX, *DawnGuard* achieves a relatively short CTT and the second-shortest testing time. Since FC-Net directly converts raw traffic into color images, it achieves the shortest FET and testing time. However, this simple representation lacks the expressiveness of our *MGF*. While *DawnGuard*'s longer FET stems from computational overheads in data augmentation and graph learning, these modules are crucial for detection accuracy improvements (Section VI-G).

G. Ablation Experiment

In this section, we evaluate the contribution of the main modules of *DawnGuard* separately, illustrated in Fig. 9. We conduct experiments with the loading ratio $\eta = 30\%$ and the imbalance ratio $\rho = 4:1$ in D_1. We also evaluate the effectiveness of *MGF* with quantitative measures.

1) Self-Adjusting Data Augmentation Module: To validate our augmentation strategy, we first remove the module entirely. This results in a significant performance degradation, with an average drop of 22.33% in F1 and 5.33% in AUC. Furthermore, we evaluate an alternative strategy that applies a static high weight to the first 30% of the traffic. While this simple weighting improves the F1 to 82.95% from the non-augmented baseline (74.11%), it remains 13.49% lower than *DawnGuard*. This substantial gap demonstrates that the effectiveness of our approach stems from generating diverse, structurally-aware samples, rather than merely biasing the model's attention towards the beginning of the flow.

2) Multi-Flow Temporal Graph Learning Module: To justify our choice of temporal graph learning, we compare *DawnGuard* against two categories of modeling paradigms: static graph neural networks (GraphSAGE [36] and GAT [37]) and fundamental Probabilistic Graphical Models (PGMs) including Hidden Markov Model (HMM) [38], Bayesian Networks [39], and Markov Random Field (MRF) [40]. The results clearly show the necessity of capturing temporal dynamics. Replacing Temporal GraphSAGE with static GraphSAGE and GAT causes the F1 to drop to 87.46% and 58.60%, respectively. Furthermore, the PGM-based models (HMM, Bayesian Networks, and MRF) exhibit limited effectiveness, achieving F1 of only 71.01%, 71.00%, and 74.92%. This performance gap demonstrates that PGMs, limited by structural assumptions and feature representation scalability, cannot effectively model the temporal-topological correlations that *DawnGuard* extracts.

3) Early-Stage Malware Detection Module: We select ViT [32] as the backbone for its superior feature extraction capabilities. To validate this choice, we conduct experiments by replacing ViT with other powerful architectures suitable for processing our 2×1216 MGF matrix: Transformer [31], ResNet50, and LSTM. All three alternatives lead to a

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
COMPARISON BETWEEN DAWNGUARD AND THE SOTA BASELINE METHODS EVALUATED IN THIS WORK

Method	Traffic Representation	Feature Scope	Model Architecture	Detection Stage	Total Malware Analyzed
FC-Net [15]	Color image	Single-flow	CNN & Meta-learning	Full-flow	18 attack types (e.g., Web Attack, DDoS, Botnet)
Kitsune [13]	Statistical features	Packet-level	Ensemble of Autoencoders	Packet-level	9 attack types (e.g., Mirai, DoS, MitM)
ST-Graph [2]	Spatio-temporal graph	Multi-flow	Random walk & Random forest	Full-flow	58 malware families (e.g., Minerd, Cryxos, PhishingSite)
CBSeq [3]	Behavior sequence	Multi-flow	Transformer	Full-channel	54 malware types (e.g., Zeus, Emotet, Miuref)
YaTC [16]	Byte-based representation	Single-flow	Masked Autoencoder	First 5 packets	10 malware types & 3 attack types (e.g., Cridex, Geodo, Flood)
DawnGuard	***MGF***	**Multi-flow**	**Temporal Graph & ViT**	**Early-stage**	**64 malware types (e.g., Dridex, Emotet, Geodo)**

notable performance decline. Specifically, their F1 decrease by 14.09%, 18.65%, and 21.01%, respectively, compared to the ViT-based *DawnGuard*. This demonstrates that ViT's global self-attention mechanism is more effective at mining high-level discriminative features from the *MGF* representations, thus enhancing the overall detection performance.

4) Effectiveness of Feature Representation: To evaluate the effectiveness of our multi-flow representation *MGF*, we compare it against three flow-based baselines: Color Image [15], ST-Graph [2], and MFR [16], excluding non-flow-based methods [3], [13]. Using 3,000 samples per category from D_1, we vectorize the initial 30% of each flow to compute the Euclidean distance between benign and malware samples (larger indicates better discriminability). The hyperparameters chosen for baselines are identical to those in [2], [15], and [16].

As shown in TABLE X, *MGF* achieves the highest average distance across all malware categories. This quantitative evidence confirms that *MGF* extracts more discriminative features than existing representations, creating a feature space where benign and malicious traffic are more separable thereby, reducing the detection difficulty in early stages.

VII. RELATED WORK

Existing malware traffic detection methods can be broadly categorized into rule-based, machine learning (ML)-based, deep learning (DL)-based, and pre-training-based methods. We summarize the multi-dimensional differences between *DawnGuard* and the representative SOTA baseline methods evaluated in this study in Table XI.

A. Rule- and ML-Based Methods

Traditional malware traffic detection methods often rely on predefined rules. Bro [11] utilizes a layered architecture to separate event engines from policy scripts for real-time detection. To handle encrypted traffic, ML-based methods focused on extracting statistical features from specific protocols (e.g., TLS/DNS) [20] or leveraging graph models. For instance, ST-Graph [2] constructs spatio-temporal graphs to analyze full flows. Other methods leverage factor graphs to infer user states [41]. Furthermore, to achieve detection without complete traffic, researchers have explored side-channel features [42], randomness tests [12], and transport-layer behaviors [43].

B. DL-Based Methods

To reduce reliance on expert knowledge, DL-based methods have been proposed for automated feature extraction. GGFAST [44] extracts characteristic sequences of message lengths to achieve classification. Kitsune [13] uses autoencoders for packet-level statistical feature analysis. To utilize multi-flow features, CBSeq [3] employs Transformers to analyze behavior sequences from full channels. Other approaches include attention-based certificate identification [45] and methods that adopt the sequential nature of packets within isolated flows for early detection [14].

C. Pre-Training-Based Methods

In recent years, pre-training frameworks have been introduced to malware traffic detection models. FC-Net [15] leverages meta-learning for few-shot detection by representing full flows as color images datasets. In another approach, YaTC [16] utilizes masked autoencoders to learn high-level representations from the first five packets.

D. Summary

In conclusion, the limitations of existing methods reflect two fundamental challenges of NIDS [46], [47], [48]: **1) Information sparsity in early-stage traffic.** Most methods rely on intra-flow features within a flow and suffer from a significant performance drop when only a small fraction of traffic is available. **2) Heterogeneity of attack patterns.** Existing methods often treat flows in isolation, failing to capture the features across contextual flows. *DawnGuard* addresses these gaps by integrating self-adjusting data augmentation, which dynamically optimizes tail-masking to focus the model on discriminative early-stage interaction phases, with multi-flow temporal graph learning to extract temporal-topological correlations for reliable detection with early-stage traffic data.

VIII. CONCLUSION

In this paper, we proposed *DawnGuard*, an effective and reliable early-stage encrypted malware traffic detection method. Specifically, *DawnGuard* utilized the temporal packet density distribution of malware traffic to achieve self-adjusting data augmentation and employed temporal graph learning to extract temporal-topological correlations among multiple flows, which formed *MGF*. By utilizing *MGF*, *DawnGuard* constructed a ViT-based detector to capture both local and global contextual relationships, thereby enabling early-stage encrypted malware traffic detection. We conducted extensive experiments to evaluate *DawnGuard*, and the results demonstrated that *DawnGuard* achieved effectiveness for early-stage

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.

encrypted malware traffic detection. In future work, we will combine *DawnGuard* with unsupervised learning to improve the performance of early-stage unknown encrypted malware traffic detection. Furthermore, we plan to deploy *DawnGuard* in an edge gateway for analysis on large-scale ISP traffic to further validate its robustness in high-throughput environments.

REFERENCES

[1] E. Rescorla, *SSL and TLS: Designing and Building Secure Systems*, vol. 1. Reading, MA, USA: Addison-Wesley, 2001.

[2] Z. Fu et al., "Encrypted malware traffic detection via graph-based network analysis," in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2022, pp. 495–509.

[3] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lü, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Secur.*, pp. 5011–5025, Aug. 2023.

[4] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023.

[5] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.

[6] C. V. Wright, F. Monrose, G. M. Masson, and P. Chan, "On inferring application protocol behaviors in encrypted network traffic," *JMLR*, vol. 7, pp. 2745–2769, Dec. 2006.

[7] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[8] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. USENIX Secur.*, Aug. 2023, pp. 607–624.

[9] M. Shen et al., "Swallow: A transfer-robust website fingerprinting attack via consistent feature learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 1574–1588.

[10] (2024). *WatchGuard's Threat Lab Analyzes the Latest Malware and Internet Attacks*. [Online]. Available: https://www.watchguard.com/wgrd-resource-center/security-report-q4-2024

[11] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Comput. Netw.*, vol. 31, nos. 23–24, pp. 2435–2463, Dec. 1999.

[12] W. Niu, Z. Zhuo, X. Zhang, X. Du, G. Yang, and M. Guizani, "A heuristic statistical testing based approach for encrypted network traffic identification," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3843–3853, Apr. 2019.

[13] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018.

[14] T. E. T. Djaidja, B. Brik, S. Mohammed Senouci, A. Boualouache, and Y. Ghamri-Doudane, "Early network intrusion detection enabled by attention mechanisms and RNNs," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7783–7793, 2024.

[15] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020.

[16] R. Zhao et al., "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM TON*, vol. 32, no. 3, pp. 2012–2025, Jan. 2024.

[17] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, and L. Zhu, "Robust detection of malicious encrypted traffic via contrastive learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4228–4242, 2025.

[18] M. Cen, F. Jiang, X. Qin, Q. Jiang, and R. Doss, "Ransomware early detection: A survey," *Comput. Netw.*, vol. 239, Feb. 2023, Art. no. 110138.

[19] Stratosphere.(2015). *Stratosphere Laboratory Datasets*. [Online]. Available: https://www.stratosphereips.org/datasets-overview

[20] B. Anderson and D. McGrew, "Identifying encrypted malware traffic with contextual flow data," in *Proc. ACM Workshop Artif. Intell. Secur.*, Oct. 2016, pp. 35–46.

[21] G. Jacob, R. Hund, C. Kruegel, and T. Holz, "JACKSTRAWS: Picking command and control connections from bot traffic," in *Proc. USENIX Secur. 11*, 2011, p. 29.

[22] F. Tegeler, X. Fu, G. Vigna, and C. Kruegel, "BotFinder: Finding bots in network traffic without deep packet inspection," in *Proc. 8th Int. Conf. Emerg. Netw. Experiments Technol.*, Dec. 2012, pp. 349–360.

[23] X. Deng, Q. Li, and K. Xu, "Robust and reliable early-stage website fingerprinting attacks via spatial–temporal distribution analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 1997–2011.

[24] A. Alqahtani, M. Gazzan, and F. T. Sheldon, "A proposed crypto-ransomware early detection(CRED) model using an integrated deep learning and vector space model approach," in *Proc. 10th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2020, pp. 0275–0279.

[25] S. Rahmat, Q. Niyaz, A. Mathur, W. Sun, and A. Y. Javaid, "Network traffic-based hybrid malware detection for smartphone and traditional networked systems," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, Oct. 2019, pp. 0322–0328.

[26] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, "Fine-grained webpage fingerprinting using only packet length information of encrypted traffic," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2046–2059, 2021.

[27] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, Apr. 2022, pp. 1–9.

[28] R. W. Schafer, "What is a Savitzky-Golay filter? [lecture notes]," *IEEE Signal Process. Mag.*, vol. 28, no. 4, pp. 111–117, Jul. 2011.

[29] D. Koukis, S. Antonatos, D. Antoniades, E. P. Markatos, and P. Trimintzios, "A generic anonymization framework for network traffic," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2006, pp. 2302–2309.

[30] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. Cambridge, MA, USA: MIT Press, 2009.

[31] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 6000–6010.

[32] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2020.

[33] R. Zamir, "A proof of the Fisher information inequality via a data processing argument," *IEEE TIT*, vol. 44, no. 3, pp. 1246–1250, May 1998.

[34] W. Gao, S. Kannan, S. Oh, and P. Viswanath, "Estimating mutual information for discrete-continuous mixtures," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5988–5999.

[35] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.

[36] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NIPS*, vol. 30, 2017, pp. 1024–1034.

[37] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[38] L. R. Rabiner and B. Juang, "An introduction to hidden Markov models," *IEEE ASSP Mag.*, vol. ASSPM-3, no. 1, pp. 4–16, Jan. 1986.

[39] F. V. Jensen, "Bayesian networks," *Encyclopedia Statist. Quality Rel.*, vol. 1, no. 3, pp. 307–315, 2009.

[40] S. Z. Li, *Markov Random Field Modeling in Image Analysis*. Cham, Switzerland: Springer, 2009.

[41] P. Cao, E. Badger, Z. Kalbarczyk, R. Iyer, and A. Slagell, "Preemptive intrusion detection: Theoretical framework and real-world measurements," in *Proc. Symp. Bootcamp Sci. Secur.*, Apr. 2015, pp. 1–12.

[42] G. Stergiopoulos, A. Talavari, E. Bitsikas, and D. Gritzalis, "Automatic detection of various malicious traffic using side channel features on TCP packets," in *Proc. ESORICS*, 2018, pp. 346–362.

[43] Z. Berkay Celik, R. J. Walls, P. McDaniel, and A. Swami, "Malware traffic detection using tamper resistant features," in *Proc. MILCOM - IEEE Mil. Commun. Conf.*, Oct. 2015, pp. 330–335.

[44] J. Piet, D. Nwoji, and V. Paxson, "GGFAST: Automating generation of flexible network traffic classifiers," in *Proc. ACM SIGCOMM Conf.*, 2023, pp. 850–866.

[45] I. Torroledo, L. D. Camacho, and A. C. Bahnsen, "Hunting malicious TLS certificates with deep neural networks," in *Proc. 11th ACM Workshop Artif. Intell. Secur. (CCS)*, S. Afroz, B. Biggio, Y. Elovici, D. Freeman, and A. Shabtai, Eds., Toronto, ON, Canada: ACM, Oct. 2018, pp. 64–73.

[46] J. Sowa et al., "Post-quantum cryptography (PQC) network instrument: Measuring PQC adoption rates and identifying migration pathways," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, Sep. 2024, pp. 1835–1846.

[47] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 305–316.

[48] L. Yang et al., "True attacks, attack attempts, or benign triggers? An empirical measurement of network alerts in a security operations center," in *Proc. USENIX Secur.*, 2024, pp. 1525–1542.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:44:55 UTC from IEEE Xplore. Restrictions apply.